Doctoral Dissertation

An Intent-Aware Quick Debugging Assistant for Solidity

Inseong Jeon

Department of Computer Science

Graduate School

Korea University

February 2027

An Intent-Aware Quick Debugging Assistant for Solidity

by
Inseong Jeon

---

under the supervision of Professor Hoh Peter In

A dissertation submitted in partial fulfillment of
the requirements for the degree of
Doctor of Philosophy

Department of Computer Science

Graduate School
Korea University

October 2026

The dissertation of Inseong Jeon has been approved
by the dissertation committee in partial fulfillment
of the requirements for the degree of
Doctor of Philosophy

December 2026

_____

Committee Chair:

_____

Committee Member:

_____

Committee Member:

_____

Committee Member:

_____

Committee Member:

An Intent-Aware Quick Debugging Assistant for Solidity

Inseong Jeon
Department of Computer Science
under the supervision of Professor Hoh Peter In

Abstract

TODO: English abstract.

Keywords: smart contract; Solidity; developer intent; abstract interpretation; interval domain; incremental analysis; debugging

국문 제목

전 인 성
컴 퓨 터 학 과
지 도 교 수: 인 호

국문 초록

TODO: 국문 초록.

중심어 : 스마트 컨트랙트; 솔리디티; 개발자 의도; 추상해석; 구간 도메인; 증분 분석; 디버깅

TODO: dedication (optional; no heading, centered).

Preface

TODO: preface. It must state, at minimum:

1. which results were obtained in collaboration, the nature and proportion of others' contributions, and which portions are claimed as original;
2. funding sources and other contributors;
3. which contents have been published or submitted for publication, and the contributions of every author of those multi-authored works (SOLQDEBUG, INTENTCHECKER);
4. optionally: background, motivation, target readership.

Acknowledgment

TODO: acknowledgment.

Table of Contents

Abstract i

국문초록 ii

Preface iv

Acknowledgment v

Table of Contents vi

List of Tables ix

List of Figures x

List of Listings xi

Nomenclature xii

1 Introduction 1

1.1 Motivation 1

1.2 Proposed Techniques 1

1.3 Contributions 1

1.4 Outline 1

2 Background and Problem Definition 2

2.1 Solidity Execution Model and Debugging Workflow 2

2.2 Numeric Logic Errors in Solidity 5
2.3 Problem Definition 8
2.3.1 Transaction-Dependent and Bytecode-Dependent Debugging . . . 8
2.3.2 Predefined Detection Criteria and Limited Support for Developer Intent 10
2.4 Overview of the Proposed Approach 11
2.4.1 Transaction-Independent Source-Level Debugging 11
2.4.2 Development-Time Intent Validation 12

3 The Design of IQDebug **13**
3.1 Overview 14
3.2 Source-Level Range-Based Analysis 14
3.2.1 Source-Level Program Representation 14
3.2.2 Range-Based Debugging Inputs 14
3.2.3 Abstract Interpretation of Program States 14
3.3 Intent-Aware Validation 14
3.3.1 Developer-Authored Intent Model 14
3.3.2 Intent Validation over Abstract States 14
3.4 Interactive Debugging over Abstract Program States 14
3.4.1 Integration of Intent Validation into the Debugging Workflow . . 14
3.4.2 Debugging Navigation Model 14
3.4.3 Step-Based Navigation 14
3.4.4 Branch and Loop Navigation 14
3.4.5 Breakpoint and Continue 14
3.5 Implementation 14
3.5.1 System Implementation 14
3.5.2 Analysis Engine 14
3.5.3 Debugging Interface 14

4 Evaluation **15**
4.1 Experimental Setup 15
4.2 RQ1: Intent Validation Capability 15

4.3 Specification and Analysis Cost . . . 15
4.3.1 Specification Cost . . . 15
4.3.2 Validation and Interactive Analysis Cost . . . 15
4.4 RQ3: Comparison with Existing Detection Tools . . . 15
4.5 Threats to Validity . . . 15

5 Discussion **16**
5.1 Design Trade-offs of Range-Based Debugging . . . 16
5.2 Practical Guidelines for Debug Inputs and Intent Specification . . . 16
5.3 Current Limitations . . . 16

6 Related Work **17**
6.1 Solidity Debugging and Development Environments . . . 17
6.2 Intent- and Specification-Based Smart Contract Analysis . . . 17
6.3 Abstract Interpretation and Interactive Static Analysis . . . 17

7 Conclusion **18**

Bibliography **19**

A Typesetting Reference **20**
A.1 Solidity listing with intent annotations . . . 20
A.2 Algorithm . . . 21
A.3 Theorems and inference rules . . . 21
A.4 Figures and tables . . . 22

Index **23**

List of Tables

2.1 Representative single-transaction numeric logic error patterns from prior work, classified by abstraction level. 7

A.1 Text width on B5 is 136 mm — wide tables need `sidewaystable` 22

List of Figures

2.1 Compilation in a conventional Solidity debugging workflow. 3
2.2 Contract deployment in a conventional Solidity debugging workflow. . . . 3
2.3 State initialization and target-function execution in a conventional Solidity debugging workflow. 4
2.4 Transaction-based bytecode debugging in a conventional Solidity debugging workflow. 5
2.5 Example requiring state preparation before debugging the target function. 9

A.1 Korea University global symbol 22
A.2 A minimal control-flow graph 22

List of Listings

A.1 `@During` and `@Post` annotations on a transfer 20

Nomenclature

Intv	interval abstract domain
Store	abstract store (variable $\mapsto$ abstract value)
Env	environment
$\sqcup$	join (least upper bound)
$\sqcap$	meet (greatest lower bound)
∇	widening operator
$\triangle$	narrowing operator
$[\![\cdot]\!]^\sharp$	abstract semantics (transfer function)
$\top$, $\bot$	top and bottom of the abstract domain

Abbreviation	
CFG	Control-Flow Graph
SESE	Single-Entry Single-Exit (region)
AI	Abstract Interpretation
IR	Intermediate Representation

Chapter 1. Introduction

1.1 Motivation

1.2 Proposed Techniques

1.3 Contributions

1.4 Outline

Chapter 2. Background and Problem Definition

2.1 Solidity Execution Model and Debugging Workflow

Solidity smart contracts execute in the Ethereum Virtual Machine (EVM), a state-based execution environment in which program execution is driven by transactions. Unlike conventional applications that can often be executed directly from source code within an integrated development environment (IDE), Solidity contracts typically undergo several stages before their runtime behavior can be inspected. A conventional Solidity debugging workflow therefore involves compilation, deployment, state initialization and transaction execution, and post-execution debugging.

Step 1: Compile. A Solidity source file is first compiled into EVM bytecode and its associated application binary interface (ABI). Figure 2.1 illustrates this step using Remix IDE. The resulting bytecode constitutes the executable representation of the contract, while the ABI provides the interface through which users and other contracts invoke its externally accessible functions.

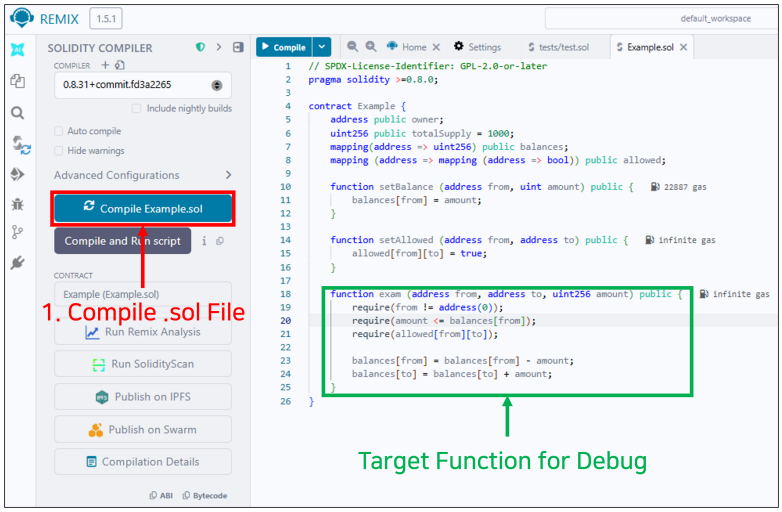


Figure 2.1: Compilation in a conventional Solidity debugging workflow.

Step 2: Deploy. Before a contract can be executed, the compiled bytecode is deployed to a blockchain or a local blockchain environment. As illustrated in Figure 2.2, deployment creates a contract instance and invokes its constructor to initialize the contract state. The deployed contract subsequently maintains persistent state across transactions.

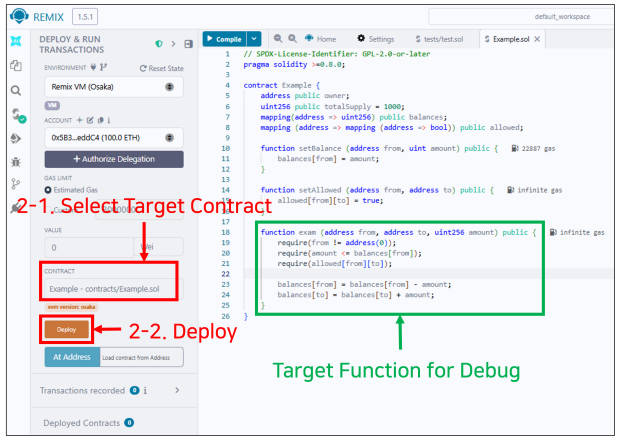


Figure 2.2: Contract deployment in a conventional Solidity debugging workflow.

Step 3: Initialize State and Execute. To inspect a target function, a developer must prepare an execution state in which the function can be invoked. This preparation may

require calling other functions in advance to initialize contract state or satisfy conditions enforced by constructs such as `require`. Once the required state has been established, the developer supplies concrete function arguments and executes the target function as a transaction. Figure 2.3 shows an example of this process in Remix.

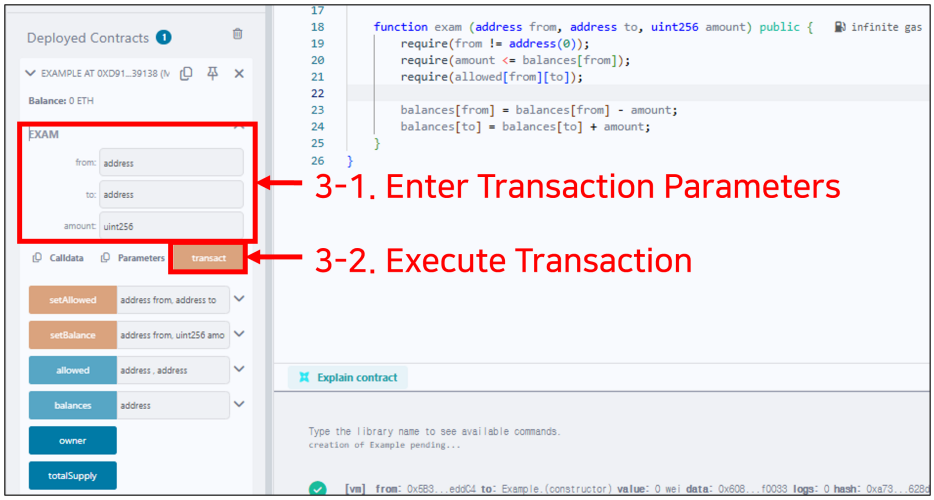


Figure 2.3: State initialization and target-function execution in a conventional Solidity debugging workflow.

Step 4: Debug. After the target transaction has been executed, conventional Solidity debugging tools can expose its execution trace. For example, Remix allows developers to inspect the executed transaction and step through the corresponding EVM instructions, as illustrated in Figure 2.4. The debugger provides information such as the current opcode, stack and memory contents, storage values, and the relationship between bytecode instructions and source-code locations.

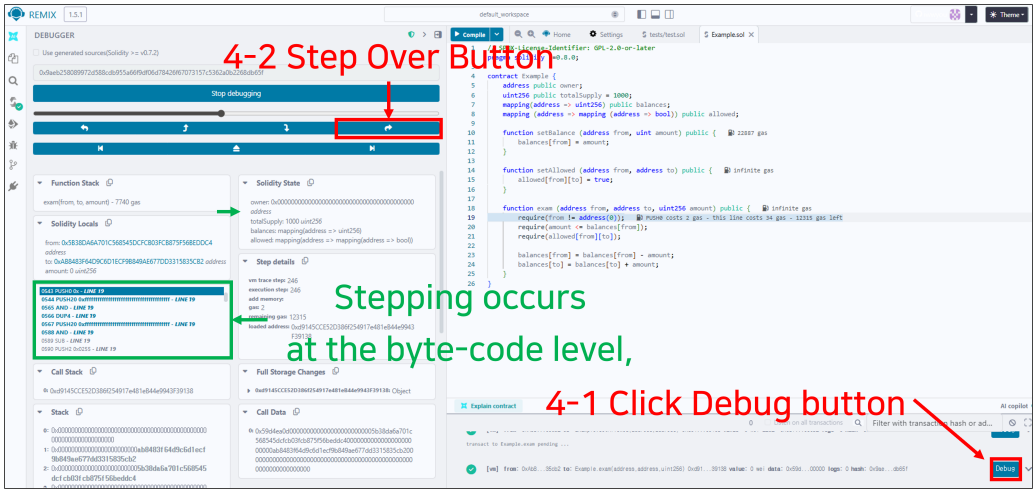


Figure 2.4: Transaction-based bytecode debugging in a conventional Solidity debugging workflow.

This workflow differs fundamentally from the edit–execute–debug cycle commonly available for traditional software. In conventional Solidity debugging, the behavior to be inspected is materialized through a concrete execution: the contract is compiled and deployed, the required state is established, concrete inputs are supplied, and the resulting transaction trace is subsequently examined. Accordingly, the observable debugging context is closely coupled to executable artifacts, initialized contract state, and a particular transaction execution. These characteristics become particularly relevant when debugging support is considered during source-code development, where program behavior may need to be inspected before or while a complete concrete execution context is being prepared.

2.2 Numeric Logic Errors in Solidity

Recent studies have reported a variety of numeric-related weaknesses in smart contracts [1, 4–7]. Unlike failures that trigger compilation errors, runtime exceptions, or transaction reverts, some numerical defects allow a transaction to complete successfully while producing values that are inconsistent with the intended behavior of the contract. Following prior work, we refer to such defects as *numeric logic errors*.

Definition 2.1 (Numeric Logic Error)**.** A numeric logic error is a bug that allows a transaction to complete successfully without compile-time or runtime exceptions, but violates the developer's intended relationships among numerical values during function execution.

This definition emphasizes an important characteristic of numeric logic errors. Whether a numerical computation is erroneous cannot always be determined solely from the fact that execution succeeds or from the syntactic form of the computation. Instead, correctness depends on whether the resulting numerical relationships conform to the behavior intended by the developer. For example, two arithmetic expressions may both be syntactically valid and execute without reverting, while only one preserves the intended relationship among balances, exchange amounts, rewards, or accounting variables. Thus, the intended numerical relationship constitutes part of the criterion by which such program behavior is judged.

Numeric logic errors can be distinguished according to the execution scope in which they manifest. *Single-transaction errors* arise within one atomic transaction. The relevant computations may span multiple internal or external function calls, but the behavior leading to the erroneous numerical result is contained within a single transaction. In contrast, *multiple-transaction errors* emerge only through a sequence of transactions and therefore depend on state changes accumulated across multiple function invocations. Examples include Risky First Deposit and Price Oracle Manipulation [5, 6], whose effects become observable only after contract state has evolved over multiple transactions.

This dissertation primarily focuses on single-transaction numeric logic errors, including cases in which a transaction crosses contract boundaries through external calls. This scope is consistent with the development-time analysis considered throughout the dissertation: the behavior of interest is bounded by one transaction and can therefore be analyzed with respect to the source-level execution being examined. Numeric properties that inherently span arbitrary transaction sequences require reasoning about inter-transaction dependencies and contract-level invariants and are left outside the primary scope of this work.

Within the single-transaction scope, representative numeric logic errors can be organized according to the abstraction level at which the erroneous behavior arises. Table 2.1

Table 2.1: Representative single-transaction numeric logic error patterns from prior work, classified by abstraction level.

Pattern	Description
Value-level errors: incorrect numerical values arise directly from computations.	
Div In Path [1]	Division results used in conditions may alter control-flow decisions because of integer truncation.
Exchange Problem [1]	Rounding behavior in exchange calculations may produce an unintended output, such as zero output for a valid input or an imbalanced exchange.
Precision Loss Trend [1]	Repeated or improperly ordered integer arithmetic may systematically introduce truncation that favors one side of a computation.
Minor Amount Retention [1]	Integer division in distribution logic may leave numerical remainders unallocated.
Algorithm-level errors: incorrect numerical behavior arises from the structural arrangement of operations.	
Operator Order Issue [1]	Different arithmetic evaluation orders can produce different integer results because intermediate truncation is not preserved across algebraically equivalent expressions.
Erroneous Accounting / Wrong Interest Rate Order [5, 6]	Balance calculations use stale or prematurely updated interest-rate information, resulting in an incorrect numerical outcome.
Erroneous Accounting / Wrong Checkpoint Order [5, 6]	State updates and checkpoints are performed in an incorrect order, causing subsequent reward or accounting calculations to use unintended values.
Inconsistent State Updates [6]	A required state update is omitted, leaving subsequent computations to operate on stale numerical state.
Greedy Contract [4]	The contract accepts funds but lacks the corresponding operation needed to release them, resulting in an unintended numerical state.

summarizes representative patterns reported in prior studies.

Value-level errors arise when the computation directly produces an unintended numerical value. Typical examples include the use of an incorrect operator or identifier, precision loss caused by integer arithmetic, or an arithmetic expression whose result differs from the value intended by the developer. The erroneous behavior is therefore observable primarily through the values produced by the computation.

Algorithm-level errors, in contrast, arise from the structural arrangement of operations. Examples include executing otherwise valid operations in an incorrect order, omitting a required state update, or failing to perform a necessary procedure call. Although

their underlying cause is structural, their effect may still become observable as an unintended relationship among numerical values.

The following sections build on these characteristics of numeric logic errors to formulate the limitations of existing debugging and analysis approaches and to motivate source-level debugging with developer-authored intent for development-time analysis.

2.3 Problem Definition

The preceding background highlights two related limitations in Solidity development. First, conventional debugging depends on concrete transactions and bytecode execution, which limits direct inspection of program behavior during source-code development. Second, although numeric logic errors are defined with respect to developer-intended numerical relationships, existing analysis workflows generally provide limited support for developers to directly supply such intent as an analysis criterion. The following subsections describe these two problems in detail.

2.3.1 Transaction-Dependent and Bytecode-Dependent Debugging

Conventional Solidity debugging is tightly coupled to concrete execution. As described in Section 2.1, a developer typically compiles the source code, deploys the resulting bytecode, prepares the required contract state, executes a transaction with concrete inputs, and then inspects the resulting execution trace. This workflow introduces two closely related difficulties for development-time debugging.

Repeated preparation of concrete execution contexts. A target function frequently depends on contract state that has been established before its invocation. Consequently, debugging a particular behavior may require developers to execute additional transactions in advance to initialize balances, counters, reserves, permissions, or other state variables.

Figure 2.5 shows a simple example. The `withdraw` function contains a `require` condition that prevents execution from proceeding when the caller's balance is insufficient. Therefore, the balance-update statement can be reached only after an appropriate balance has been established. A developer who wants to debug this statement must first execute `deposit` to prepare the required contract state before invoking `withdraw`. If the target

```
contract Example {
mapping(address => uint256) balances;

```
function deposit() external payable {
    balances[msg.sender] += msg.value;
}

function withdraw(uint256 amount) external {
    require(balances[msg.sender] >= amount);
    balances[msg.sender] -= amount;
    payable(msg.sender).transfer(amount);
}
```

}
```

Figure 2.5: Example requiring state preparation before debugging the target function.

source code or debugging scenario changes, this preparation may need to be repeated. The same issue arises when developers want to inspect the behavior of a function under different input conditions. Each concrete execution represents only one particular combination of function arguments and contract state. Exploring alternative behaviors therefore requires additional executions with different inputs or state configurations. This repeated preparation increases the cost of iterative debugging by requiring developers to repeatedly reconstruct concrete execution contexts.

Dependence on post-execution bytecode traces. Detailed debugging information is generally obtained only after a transaction has been executed. The debugger then presents the execution through EVM-level information such as opcodes, stack values, memory, storage, and source mappings. Although this information accurately represents the executed transaction, it is derived from compiled bytecode rather than directly from the source-level program state in which the developer is reasoning.

This creates a mismatch between the level at which developers write and understand Solidity programs and the level at which conventional debugging exposes execution details. A single Solidity statement may correspond to multiple EVM instructions, and the information relevant to a source-level variable or expression may be distributed across several low-level operations. As a result, identifying how numerical values evolve

through a source-level computation may require additional interpretation of the underlying execution trace.

These characteristics make conventional debugging inherently dependent on the availability of a complete concrete execution. During development, however, developers may need to inspect how values can evolve before constructing every concrete transaction required to reproduce a specific behavior. The first problem addressed in this dissertation is therefore how to provide source-level debugging information without requiring repeated transaction execution and bytecode-level tracing for each debugging scenario.

2.3.2 Predefined Detection Criteria and Limited Support for Developer Intent

Static analysis, symbolic execution, fuzzing, type-based analysis, and learning-assisted approaches have been widely used to detect vulnerabilities and logic errors in smart contracts. Many of these approaches identify suspicious behavior using criteria predefined by the analysis, such as vulnerability patterns, type constraints, arithmetic rules, inferred invariants, or previously characterized bug scenarios.

Such criteria are effective when the target defect corresponds to a property already represented by the analysis. For example, a tool designed to detect a particular arithmetic pattern can report program locations that match that pattern, while a type-based analysis can identify operations that violate its predefined type rules. However, numeric logic errors are defined relative to the numerical relationships intended by the developer. Two computations may therefore both satisfy generic analysis rules while differing in whether they conform to the intended behavior of a particular contract. In such cases, the relevant correctness criterion is contract-specific and may be known primarily to the developer. Existing analysis workflows, however, generally provide limited support for developers to directly supply such numerical intent as an analysis criterion. Extending the set of predefined rules or patterns can increase coverage, but it does not provide a direct mechanism for expressing contract-specific numerical intent.

The stage at which analysis is performed introduces an additional limitation. Many existing approaches are primarily applied to complete implementations, deployed arti-

facts, or test executions. In these workflows, the analysis is performed after executable program behavior has been constructed and is evaluated against criteria already defined by the tool or analysis method. This differs from a development-time workflow in which developer-authored numerical intent can be provided together with the evolving source code and used directly as an analysis criterion.

2.4 Overview of the Proposed Approach

To address the two limitations identified in Section 2.3, this dissertation combines two complementary approaches.

2.4.1 Transaction-Independent Source-Level Debugging

To reduce the dependence on concrete transactions and bytecode-level traces, this dissertation adopts transaction-independent source-level debugging based on interval analysis.

Instead of executing a function with a single concrete input and contract state, the approach represents relevant numerical values as intervals and analyzes how these abstract values evolve through the source-level program. For example, rather than repeatedly executing a function with several individual input values, a developer can specify a range of values of interest. The analysis represents this range using the interval abstract domain, propagates it through the program, and derives abstract program states at relevant source-code locations. These abstract states summarize possible numerical values without requiring a separate concrete transaction for every value represented by the interval.

This approach also changes the level at which debugging information is presented. Rather than requiring developers to primarily inspect EVM instructions after transaction execution, program behavior can be examined in relation to source-level statements and variables. The resulting abstract states can therefore serve as a debugging context in which developers inspect how values evolve across statements, branches, function calls, and loops.

The resulting debugging process does not reproduce a single concrete execution trace. Instead, it summarizes a set of possible executions represented by the supplied ranges

through interval-based abstract analysis. This enables program behavior to be inspected without repeatedly preparing every corresponding concrete execution context, while retaining source-level information needed for interactive debugging.

2.4.2 Development-Time Intent Validation

Transaction-independent source-level debugging provides information about how program values may evolve, but the resulting abstract states alone do not determine whether the analyzed behavior is correct. As discussed in Section 2.2, numeric logic errors are defined relative to numerical relationships intended by the developer. This dissertation therefore complements source-level debugging with a mechanism that allows developers to explicitly provide such relationships as analysis criteria.

The approach associates developer-authored numerical intent with relevant source-level program states. Instead of relying exclusively on vulnerability patterns or other detection criteria predefined by an analysis tool, the specified intent describes relationships that the developer expects program values to satisfy. The analysis can then evaluate the abstract program states derived through interval-based analysis against these developer-provided criteria. Providing intent during development also allows the analysis criterion to evolve together with the source code. Developers can specify numerical relationships relevant to the behavior currently being implemented or inspected and validate them within the same source-level analysis context used for debugging.

In this way, source-level debugging provides the program states to be inspected, while developer-authored intent provides an explicit criterion for determining whether the analyzed numerical behavior conforms to the intended behavior.

Chapter 3. The Design of IQDebug

3.1 Overview

3.2 Source-Level Range-Based Analysis

3.2.1 Source-Level Program Representation

3.2.2 Range-Based Debugging Inputs

3.2.3 Abstract Interpretation of Program States

3.3 Intent-Aware Validation

3.3.1 Developer-Authored Intent Model

3.3.2 Intent Validation over Abstract States

3.4 Interactive Debugging over Abstract Program States

3.4.1 Integration of Intent Validation into the Debugging Workflow

3.4.2 Debugging Navigation Model

3.4.3 Step-Based Navigation

3.4.4 Branch and Loop Navigation

3.4.5 Breakpoint and Continue

3.5 Implementation

3.5.1 System Implementation

3.5.2 Analysis Engine

3.5.3 Debugging Interface

Chapter 4. Evaluation

4.1 Experimental Setup

4.2 RQ1: Intent Validation Capability

4.3 Specification and Analysis Cost

4.3.1 Specification Cost

4.3.2 Validation and Interactive Analysis Cost

4.4 RQ3: Comparison with Existing Detection Tools

4.5 Threats to Validity

Chapter 5. Discussion

5.1 Design Trade-offs of Range-Based Debugging

5.2 Practical Guidelines for Debug Inputs and Intent Specification

5.3 Current Limitations

Chapter 6. Related Work

6.1 Solidity Debugging and Development Environments

6.2 Intent- and Specification-Based Smart Contract Analysis

6.3 Abstract Interpretation and Interactive Static Analysis

Chapter 7. Conclusion

Bibliography

[1] Jiachi Chen et al. NumScout: Unveiling numerical defects in smart contracts using LLM-pruning symbolic execution. *IEEE Transactions on Software Engineering*, 2025. doi: 10.1109/TSE.2025.3555622.

[2] Consensys Diligence. Scribble. https://diligence.security/scribble/, 2026. Accessed April 2026.

[3] Inseong Jeon et al. SolQDebug: Debug Solidity quickly for interactive immediacy in smart contract development. *Automated Software Engineering*, 33(3):86, 2026. doi: 10.1007/s10515-026-00618-9.

[4] Majd Soud, Grischa Liebel, and Mohammad Hamdaqa. A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. *Empirical Software Engineering*, 29(1):13, 2024. doi: 10.1007/s10664-023-10398-5.

[5] Yuqiang Sun et al. GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13, 2024. doi: 10.1145/3597503.3639117.

[6] Zhuo Zhang et al. Demystifying exploitable bugs in smart contracts. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 615–627, 2023. doi: 10.1109/ICSE48619.2023.00061.

[7] Liyi Zhou et al. SoK: Decentralized finance (DeFi) attacks. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*, pages 2444–2461, 2023. doi: 10.1109/SP46215.2023.10179435.

Appendix A. Typesetting Reference

A.1 Solidity listing with intent annotations

```
function withdraw(uint256 amount) public {
    // @During balances[msg.sender](Before) >= amount
    require(balances[msg.sender] >= amount);
    balances[msg.sender] -= amount;
    // @Post balances[msg.sender](Exit) == balances[msg.sender](Entry) - amount
}
```

Listing A.1: `@During` and `@Post` annotations on a transfer

Listing A.1 shows snapshot-qualified references (`(Entry)`, `(Exit)`, `(Before)`, `(After)`, `(Assign)`).

A.2 Algorithm

Algorithm 1 Fixpoint iteration over a function CFG

Require: function CFG $G = (N, E)$, initial store σ_0

Ensure: $Store^\sharp : N \to Intv$

1: $W \leftarrow \{ n_{entry} \}$
2: **while** $W \neq \emptyset$ **do**
3: pop n from W
4: $in \leftarrow \bigsqcup_{p \in pred(n)} out[p]$
5: $out' \leftarrow [\![n]\!]^\sharp(in)$
6: **if** $out' \not\sqsubseteq out[n]$ **then**
7: $out[n] \leftarrow out[n] \nabla out'$
8: $W \leftarrow W \cup succ(n)$
9: **end if**
10: **end while**
11: **return** out

A.3 Theorems and inference rules

Definition A.1 (Abstract store)**.** An abstract store $\sigma^\sharp \in Store$ maps each variable to an element of $Intv$.

Theorem A.2 (Soundness)**.** *For every concrete store* $\sigma \in \gamma(\sigma^\sharp)$, $[\![c]\!]\sigma \in \gamma([\![c]\!]^\sharp\sigma^\sharp)$.

Proof. TODO. □

Theorem A.2 depends on Definition A.1.

$$\textsc{Assign}\ \frac{[\![e]\!]^\sharp\sigma^\sharp = v^\sharp}{[\![x := e]\!]^\sharp\sigma^\sharp = \sigma^\sharp[x \mapsto v^\sharp]} \qquad \textsc{Join}\ \frac{[\![s_1]\!]^\sharp\sigma^\sharp = \sigma_1^\sharp \qquad [\![s_2]\!]^\sharp\sigma^\sharp = \sigma_2^\sharp}{[\![\text{if } b \text{ then } s_1 \text{ else } s_2]\!]^\sharp\sigma^\sharp = \sigma_1^\sharp \sqcup \sigma_2^\sharp}$$

A.4 Figures and tables



Figure A.1: Korea University global symbol

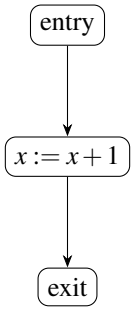


Figure A.2: A minimal control-flow graph

Table A.1: Text width on B5 is 136 mm — wide tables need `sidewaystable`

Metric	**Value**	**Note**
Paper width	176 mm	B5
Text width	136 mm	after the 20 mm side margins

Figures A.1 and A.2 and Table A.1 are cross-referenced with `cleveref`. Citations go through `natbib`: [3] and Consensys Diligence [2]. Indexed terms appear in the index.

Index

abstract interpretation, 22

index, 22